

Two Sum: Check if a pair with given sum exists in Array

Given an array of integers $arr[]$ and an integer $target$.

1st variant: Return Yes if there exist two numbers such that their sum is equal to the target. Otherwise, return No.

2nd variant: Return indices of the two numbers such that their sum is equal to the target. Otherwise, we will return $[-1, -1]$.

Ex: $N=5$, $arr[] = \{2, 6, 5, 8, 11\}$, $target = 14$
 $arr[1] + arr[3] = 14$

Op: 1st variant: Yes
2nd variant: $[1, 3]$

Ex: $N=5$, $arr[] = \{2, 6, 5, 8, 11\}$, $target = 15$

Op: 1st variant: No
2nd variant: $[-1, -1]$

Brute Force

1. First loop selects the index (i) of the array one by one.
2. For every index i , we will traverse through the remaining array using another loop (j) to find the other number such that the sum is equal to the target (i.e. $arr[i] + arr[j] = target$)

TC: $O(N^2)$

SC: $O(1)$

i	j	target
4	7	11
4	1	5
4	9	13
7	1	8

4, 7, 1, 9
i j

target = 8

Idea 1 Using Hashmap

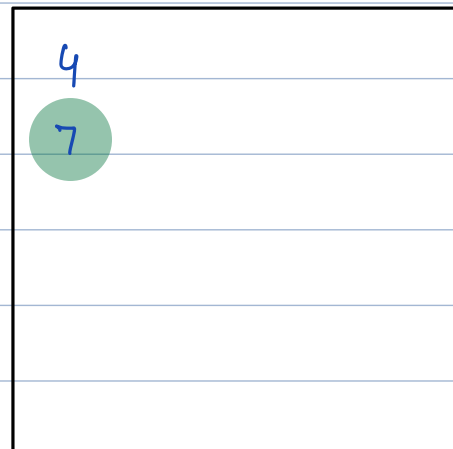
- For every i , check if $(target - arr[i])$ exists in the HashMap.
 → element exists 1. return YES
 2. will return the current index and index found
- If that element does not exist, store cur element in the HashMap along with its index.
- If no such pair exists, return "No" or $\{-1, -1\}$

4 7 1 9 ; target = 8

↑
i

target - arr[i]: 4 1 7
In Map: x x ✓

return YES



HashMap

```
class Solution {  
  
    // Variant 1: Check if two numbers sum to target using hashing  
  
    public String twoSumExists(int[] arr, int target) {  
  
        HashMap<Integer, Integer> map = new HashMap<>();  
  
        // Iterate over all elements  
  
        for (int i = 0; i < arr.length; i++) {  
  
            int complement = target - arr[i];  
  
            // Check if complement exists in map  
  
            if (map.containsKey(complement)) {  
  
                return "YES"; // Pair found  
  
            }  
  
            // Store current element and its index  
  
            map.put(arr[i], i);  
  
        }  
  
        // No pair found  
  
        return "NO";  
  
    }  
  
    // Variant 2: Return indices of two numbers that sum to target using hashing  
  
    public int[] twoSumIndices(int[] arr, int target) {  
  
        HashMap<Integer, Integer> map = new HashMap<>();  
  
        for (int i = 0; i < arr.length; i++) {  
  
            int complement = target - arr[i];  
  
            // If complement found, return indices  
  
            if (map.containsKey(complement)) {  
  
                return new int[] { map.get(complement), i };  
  
            }  
  
            // Store current element and index
```

```

    map.put(arr[i], i);
}

// No pair found

return new int[] { -1, -1 };
}
}

```

TC: $O(N)$

SC: $O(N)$

- Idea 2
1. Sort the given array.
 2. Take two pointers

$l = 0$, $r = n - 1$

3. Until $l < r$

if $arr[l] + arr[r] > sum$, $r--$

if $arr[l] + arr[r] < sum$, $l++$

if $arr[l] + arr[r] == sum$, return "Yes" / result

4. If none is satisfied, return "No" or $\{-1, -1\}$

$$\begin{array}{cccc}
 0 & 1 & 2 & 3 \\
 [& 4 & , & 7 & , & 1 & , & 9 &]
 \end{array}$$

$$\Rightarrow \begin{array}{cccc}
 2 & 0 & 1 & 3 \\
 [& 1 & , & 4 & , & 7 & , & 9 &]
 \end{array}$$

$$\begin{array}{ccc}
 \uparrow & & \uparrow \\
 \downarrow & & \downarrow \\
 1 + 9 > 8 & & r-- \\
 1 + 7 == 8 & & \text{Yes}
 \end{array}$$

```
class Solution {  
  
    // Variant 1: Check if two numbers sum to target using two-pointer approach  
  
    public String twoSumExists(int[] arr, int target) {  
  
        int n = arr.length;  
  
        // Create an array of pairs [value, original_index]  
  
        int[][] numsWithIndex = new int[n][2];  
  
        // Store each element with its original index  
  
        for (int i = 0; i < n; i++) {  
  
            numsWithIndex[i][0] = arr[i]; // value  
  
            numsWithIndex[i][1] = i;      // original index  
  
        }  
  
        // Sort the array based on the value, not index  
  
        Arrays.sort(numsWithIndex, (a, b) -> Integer.compare(a[0], b[0]));  
  
        // Initialize two pointers: one at start, one at end  
  
        int left = 0, right = n - 1;  
  
        // Run loop until pointers cross  
  
        while (left < right) {  
  
            // Calculate the sum of values at pointers  
  
            int sum = numsWithIndex[left][0] + numsWithIndex[right][0];  
  
            if (sum == target) {  
  
                // Found the pair, return "YES"  
  
                return "YES";  
  
            } else if (sum < target) {  
  
                // Sum is less than target, so move left pointer right to increase sum  
  
                left++;  
  
            } else {  
  
                // Sum is greater than target, so move right pointer left to decrease sum
```

```

        right--;
    }
}

// If loop ends without returning, no pair found
return "NO";
}

// Variant 2: Return indices of two numbers that sum to target

public int[] twoSumIndices(int[] arr, int target) {
    int n = arr.length;
    int[][] numsWithIndex = new int[n][2];
    // Store element with original index
    for (int i = 0; i < n; i++) {
        numsWithIndex[i][0] = arr[i];
        numsWithIndex[i][1] = i;
    }
    // Sort by the value to apply two-pointer
    Arrays.sort(numsWithIndex, (a, b) -> Integer.compare(a[0], b[0]));
    int left = 0, right = n - 1;
    while (left < right) {
        int sum = numsWithIndex[left][0] + numsWithIndex[right][0];
        if (sum == target) {
            // Return original indices of the two numbers found
            return new int[] {numsWithIndex[left][1], numsWithIndex[right][1]};
        } else if (sum < target) {
            // Increase sum by moving left pointer forward
            left++;
        } else {
            // Decrease sum by moving right pointer backward

```

```
right--;
```

```
}
```

```
}
```

```
// No pair found
```

```
return new int[] {-1, -1};
```

```
}
```

```
}
```

TC: $O(N \log N)$

SC: $O(N)$